9a. Overview and Goals

Martin Alfaro
PhD in Economics

INTRODUCTION

In the previous chapter, we began our exploration of high performance in Julia by focusing on type stability. We now shift our attention to memory allocations, a critical aspect of performance optimization.

Memory allocations occur whenever a new object is created, involving the reservation of memory space to store its values. In particular, memory allocations on the heap, simply referred to as *memory allocations*, incur a notable cost due to the additional CPU instructions required for memory management.

From a practical standpoint, closely monitoring memory usage is essential when performance is critical. **Excessive allocation often signals inefficiency**: if two approaches exhibit large differences in memory allocation, their execution speeds are likely to differ significantly as well.

Despite this, the interplay between memory allocation and performance is more nuanced than it first appears. In fact, **reducing memory allocation is neither necessary nor sufficient for speeding up computations**. We'll present instances where the approach that allocates more memory is actually faster. This apparent paradox reflects a trade-off: while allocations introduce overhead, the resulting objects store their data in contiguous blocks of memory. Contiguity enables the CPU to access information more efficiently, sometimes offsetting the cost of allocation.

The roadmap of the chapter is as follows:

- We begin by explaining allocations and identifying allocation patterns. In particular, Section 9b defines memory allocations formally, distinguishing between allocations on the heap and the stack. With this definition, Section 9c lists which objects in Julia allocate and which don't. In particular, vectors are allocated on the heap, and so their repeated creation degrades performance. Because of this, the chapter is ultimately about using vectors appropriately. They're primarily necessary when working with large collections or objects that must be mutable or flexible in size.
- Section 9d starts the presentation of approaches to reducing memory allocations. It highlights the use of views for creating slices, instead of producing new copies.
- Section 9e introduces reductions, a technique that computes aggregate results (like summing a vector) by accumulating outputs into scalars (which don't allocate).
- Section 9f provides another technique to decrease allocations, based on lazy computations. They define the operations to be executed, deferring their computation until the results are required. This deferral allows Julia to fuse multiple steps together, eliminating temporary allocations. Section 9g extends this technique and focuses exclusively on its application to broadcasting.

- Section 9h discusses a technique that applies across programming languages, known as pre-allocation. This initializes vectors before execution begins, reusing them throughout computations. By favoring mutation over repeated object creation, pre-allocation minimizes heap allocations.
- Section 9i introduces a solution for operations involving small objects. It leverages tuples, which by design aren't allocated on the heap. Building on this idea, the package `StaticArrays` provides vectors that are implemented on top of tuples and therefore don't allocate.